MLOps Practical

Experiment 1: MLOps Project Template Builder

Aim: To create a standard folder structure for an ML project automatically using Python.

Step 1: Install Python

1. Download Python from Python official website.
2. During installation, tick Add Python to PATH.
3. Verify the installation:

```
python --version
pip --version
```

Step 2: Create a project folder

Open PowerShell and run:

```
mkdir MLOps_Practical
cd MLOps_Practical
```

Step 3: Create a virtual environment (optional but recommended)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

If activation is blocked, you can continue without activating it.

Step 4: Create the Python file

```
notepad template_builder.py
```

Paste this code and save it.

```
import osfolders = [    "data",    "models",    "notebooks",    "src",
"tests",    "reports"]for folder in folders:    os.makedirs(folder,
exist_ok=True)    print("Created:", folder)print("MLOps project structure
created successfully!")
```

Step 5: Run the program

```
python template_builder.py
```

Expected output

```
Created: data
Created: models
Created: notebooks
Created: src
Created: tests
```

```
Created: reports
MLOps project structure created successfully!
```

Step 6: Verify the folders

```
dir
```

You should see the six folders created automatically.

Viva point: `os.makedirs()` creates folders, and `exist_ok=True` prevents an error if a folder already exists.

Experiment 3: Enhanced Project Tracking with Data Version Control (DVC)

Aim: To track changes in datasets and ML project files using DVC.

Step 1: Install Git

Download Git from Git official website.

Verify:

```
git --version
```

Step 2: Create and open a project

```
mkdir DVC_Practical
cd DVC_Practical
```

Step 3: Create a virtual environment and install DVC

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install dvc
```

Verify:

```
dvc --version
```

Step 4: Initialize Git and DVC

```
git init
dvc init
```

Configure Git identity if required:

```
git config user.name "Student"
git config user.email "student@example.com"
```

Step 5: Create a sample dataset

```
notepad data.csv
```

Add the following and save:

```
Name,Marks
Amit,80
Priya,90
Rahul,75
```

Step 6: Track the dataset using DVC

```
dvc add data.csv
```

DVC creates tracking files such as `data.csv.dvc` and updates `.gitignore`.

Step 7: Save the first version in Git

```
git add data.csv.dvc .gitignore
git commit -m "Track first dataset version"
```

Step 8: Modify the dataset

Open it again:

```
notepad data.csv
```

Change Priya's marks from `90` to `95` and save.

Run:

```
dvc add data.csv
git add data.csv.dvc .gitignore
git commit -m "Update dataset version"
```

Step 9: Check the changes

```
git log --oneline
```

This displays the saved Git versions. DVC tracks the dataset content, while Git tracks the DVC metadata.

Experiment 5: Efficient ML Deployment with Docker and FastAPI

Aim: To deploy a machine learning model as an API using FastAPI and Docker.

Step 1: Install the required software

1. Install Python from Python downloads.
2. Install Docker Desktop from Docker Desktop.
3. Open Docker Desktop and wait until the Docker engine is running.

Verify:

```
python --version
docker --version
```

Step 2: Create the project

```
mkdir ML_API
cd ML_API
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install fastapi uvicorn scikit-learn
```

Step 3: Create the ML model API

```
notepad main.py
```

Paste and save:

```
from fastapi import FastAPIfrom sklearn.linear_model import
LinearRegressionapp = FastAPI()model = LinearRegression()model.fit([[1],
[2], [3], [4]], [10, 20, 30, 40])@app.get("/")def home():    return
{"message": "ML API is running"}@app.get("/predict/{x}")def predict(x:
float):    result = model.predict([[x]])    return {"input": x,
"prediction": round(float(result[0]), 2)}
```

This model learns a simple relationship: \(y = 10x\).

Step 4: Run FastAPI

```
uvicorn main:app --reload
```

Open these addresses in your browser:

- Home: http://127.0.0.1:8000/
- API documentation: http://127.0.0.1:8000/docs

On the documentation page, select `GET /predict/{x}`, click Try it out, enter `5`, and execute.

Expected response:

```
{
  "input": 5,
  "prediction": 50.0
}
```

Stop the server using `Ctrl + C`.

Step 5: Create a Dockerfile

```
notepad Dockerfile
```

Paste this code and save it. Ensure the filename is exactly `Dockerfile`, without a `.txt` extension.

```
FROM python:3.11-slim

WORKDIR /app

COPY main.py .

RUN pip install --no-cache-dir fastapi uvicorn scikit-learn

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 6: Build the Docker image

Make sure Docker Desktop is running.

```
docker build -t ml-api .
```

Step 7: Run the container

```
docker run -d -p 8000:8000 --name ml-container ml-api
```

Open the API documentation again and test the prediction endpoint.

Step 8: Check and stop the container

```
docker ps
docker stop ml-container
docker rm ml-container
```

Result: The ML model has been deployed through FastAPI and packaged into a Docker

Experiment 7: Scalable ML Pipelines with Model Registries and Feature Stores

Aim: To train a machine learning model, save it in an MLflow model registry, and understand how features are stored for reuse.

We'll use MLflow for experiment tracking and model registration, with a small local feature-store example.

Step 1: Install Python and the libraries

```
mkdir MLflow_Practical
cd MLflow_Practical
```

```
python -m venv venv
.\venv\Scripts\Activate.ps1

pip install mlflow scikit-learn pandas
```

Check installation:

```
mlflow --version
```

Step 2: Create the ML pipeline

```
notepad train.py
```

Paste and save:

```
import mlflowimport mlflow.sklearnimport pandas as pdfrom
sklearn.linear_model import LinearRegressionfrom sklearn.metrics import
mean_squared_error# Simple feature datadata = pd.DataFrame({    "hours":
[1, 2, 3, 4, 5],    "marks": [20, 35, 50, 65, 80]})X = data[["hours"]]y =
data["marks"]# Train modelmodel = LinearRegression()model.fit(X,
y)predictions = model.predict(X)mse = mean_squared_error(y, predictions)#
Track experiment and register
modelmlflow.set_tracking_uri("sqlite:///mlflow.db")mlflow.set_experiment("S
tudent_Marks_Prediction")with mlflow.start_run():
mlflow.log_param("model_type", "LinearRegression")
mlflow.log_metric("mse", mse)    mlflow.sklearn.log_model(
sk_model=model,        name="student_model",
registered_model_name="StudentMarksModel"    )print("Model trained and
registered successfully!")print("Mean Squared Error:", round(mse, 2))
```

Step 3: Run the program

```
python train.py
```

The first run may take a little time. MLflow will create a local database and tracking artifacts.

If model registration reports that the registry is unavailable, check the installed MLflow version and its error message before continuing.

Step 4: Open the MLflow dashboard

Open a second PowerShell window, go to the same project folder, activate the environment, and run:

```
cd MLflow_Practical
.\venv\Scripts\Activate.ps1
mlflow server --host 127.0.0.1 --port 5000
```

Open the MLflow dashboard.

Look for the `Student_Marks_Prediction` experiment and its recorded run. If supported by your installed MLflow version, open the registered model `StudentMarksModel` in the Models section.

Step 5: Understand the feature store

A feature store keeps prepared input values that can be reused by multiple ML models. Here is a simple local demonstration.

Create a file:

```
notepad feature_store.py
```

Paste and save:

```
import pandas as pd# Simple feature storefeatures = pd.DataFrame({
"student_id": [101, 102, 103],    "study_hours": [2, 4, 6],
"attendance": [75, 85, 95]})features.to_csv("features.csv", index=False)#
Retrieve features for one studentstore = pd.read_csv("features.csv")student
= store[store["student_id"] == 102]print("Retrieved student
features:")print(student)
```

Run:

```
python feature_store.py
```

Expected output:

```
Retrieved student features:
   student_id  study_hours  attendance
1         102            4          85
```

This CSV is only a basic feature-store demonstration, not a production-grade feature store.

Result: The ML model is trained and tracked with MLflow, the model is registered, and prepared features are saved and retrieved.

Experiment 10: Serverless Framework Implementation with Apache OpenWhisk

Aim: To create and execute a serverless function using Apache OpenWhisk.

Simple meaning: Serverless computing lets us run a function when needed without manually managing the application server.

We'll use the OpenWhisk CLI (`wsk.exe`) on Windows, since it is suitable for this practical.

Step 1: Install the requirements

1. Install Docker Desktop from Docker Desktop if you plan to run OpenWhisk locally.
2. Download or use the OpenWhisk CLI from Apache OpenWhisk CLI releases.

If you already have `wsk.exe`, you can skip downloading it.

Verify:

```
.\wsk.exe --help
```

Step 2: Configure OpenWhisk

Important: The CLI needs an OpenWhisk server address and authentication key. Installing `wsk.exe` alone is not sufficient.

If your lab provides an OpenWhisk endpoint and authentication key, configure them:

```
.\wsk.exe property set --apihost YOUR_API_HOST --auth YOUR_AUTH_KEY
```

Replace the placeholders with the actual values provided by your lab or OpenWhisk deployment.

Verify:

```
.\wsk.exe property get
```

Then test the connection:

```
.\wsk.exe action list
```

If you receive an authentication error, ask your lab instructor for the endpoint and authentication key. Do not use invented credentials.

Step 3: Create a Python function

Create a file:

```
notepad hello.py
```

Paste and save:

```
def main(args):    name = args.get("name", "Student")    return {
"message": "Hello " + name,        "platform": "Apache OpenWhisk"    }
```

This function receives input and returns a response.

Step 4: Create the serverless action

Run:

```
.\wsk.exe action create helloAction hello.py
```

Expected result: the CLI confirms that the action was created.

Step 5: Execute the action

```
.\wsk.exe action invoke helloAction --result
```

Expected response:

```
{
  "message": "Hello Student",
  "platform": "Apache OpenWhisk"
}
```

Step 6: Pass custom input

```
.\wsk.exe action invoke helloAction --param name Sakshi --result
```

The response should contain a personalized greeting.

Step 7: List and delete the action

List the actions:

```
.\wsk.exe action list
```

Delete the action after testing:

```
.\wsk.exe action delete helloAction
```

Result: A Python serverless function was created, deployed, invoked, and deleted using Apache OpenWhisk.

-